Who-Speak-AI: Secure Virtual Assistant with Speaker Recognition

Nguyễn Mạnh Cường
ID: 23127034 • nmcuong23@clc.fitus.edu.vn

Nguyễn Trần Thiên An
ID: 23127315 • nttan23@clc.fitus.edu.vn

Nguyễn Đồng Thanh
ID: 23127538 • ndthanh23@clc.fitus.edu.vn

Ho Chi Minh City, September 4, 2026

Table of Contents

I Introduction

1. Background

Voice-based virtual assistants have advanced rapidly alongside broader progress in Artificial Intelligence, evolving from simple command-response systems into sophisticated conversational agents capable of complex reasoning and task execution. Despite their growing capabilities, widespread adoption has surfaced critical concerns regarding user privacy and security.

In particular, sensitive user information such as behavioral patterns, personal preferences, and private data must be safeguarded against unauthorized access. Speaker verification (SV) determines whether a query voice matches a claimed identity and is therefore suitable for authenticating sensitive requests. Speaker identification (SID) determines which enrolled speaker is talking and supports personalization.

Beyond security, robust Speaker Verification also enables richer, more personalized user experiences. By recognizing individual speakers, a virtual assistant can tailor its responses, recommendations, and guidance to each user's unique context and history.

However, Speaker Identification in real-world settings remains a challenging task. Spoken utterances vary considerably due to factors such as background noise, recording conditions, emotional state, and dialectal differences. Moreover, models trained on data from one linguistic or acoustic domain may generalize poorly to others, underscoring the need for evaluation across diverse and representative datasets. These challenges motivate the adoption of modern deep learning architectures that can learn robust speaker representations directly from raw or lightly processed audio.

Motivated by these considerations, this project provides a concise introduction to Speaker Identification and presents a systematic analysis and comparison of several methods and models grounded in Statistical Machine Learning, with the overarching goal of advancing Speaker Verification for reliable and efficient virtual assistants.

2. Our Contributions

This work provides three key contributions:

1. **Dataset curation.** We benchmark speaker verification across two complementary corpora: the regional *ViMD* dataset, capturing Vietnamese multi-dialectal variations, and the multilingual *TidyVoice2026* corpus, assessing cross-lingual robustness under standardized verification protocols.

2. **Model benchmarking.** We benchmark two speaker embedding architectures—RawNet3 and ECAPA-TDNN—after target-domain fine-tuning on both datasets. All reported EER, minDCF, FAR, FRR, and TAR results are obtained under a shared speaker-verification protocol.

3. **Proof-of-concept application.** We develop *Who-Speak-AI*, a working voice-based virtual assistant that demonstrates secure, speaker-aware interaction. The application adopts a modular pipeline comprising Voice Activity Detection (VAD), Automatic Speech Recognition (ASR), a Large Language Model (LLM) for dialogue, and Text-to-Speech (TTS) synthesis. This end-to-end prototype illustrates how Speaker Verification can be seamlessly integrated into a practical conversational system.

3. Report Organization

The remainder of this report is organized as follows. **Section II** reviews related works on Speaker Verification and recognition. **Section III** describes the two datasets used in our experiments. **Section IV** presents the architectures of the models under study. **Section V** details the experimental setup and evaluation methodology. **Section VI** reports and analyzes the experimental results. **Section VII** introduces the Who-Speak-AI application as a proof-of-concept deployment. Finally, **Section VIII** concludes the report and discusses future directions.

II Related Works

This section surveys the key developments in speaker recognition that inform our work, organized into three areas: the evolution of speaker embedding architectures, advances in speaker enrollment strategies, and the emerging role of phonetic information in speaker modeling.

1. Speaker Embedding Architectures

Early speaker recognition systems represented speakers using statistical models of handcrafted acoustic features. The most influential of these was the **i-vector** framework [1], which employed a Gaussian Mixture Model–Universal Background Model (GMM-UBM) pipeline to compress variable-length utterances into fixed-dimensional vectors through factor analysis. Although i-vectors became a de facto standard for over a decade, their reliance on shallow generative modeling left them sensitive to channel mismatch, background noise, and speaking-style variation.

Deep neural networks have since supplanted statistical approaches. Snyder et al. introduced **x-vectors** [2], replacing the GMM back-end with a discriminatively trained neural architecture. The x-vector system feeds MFCC features through a stack of **Time Delay Neural Network (TDNN)** layers [3] that model temporal context at the frame level, aggregates them via a statistics pooling layer, and classifies speakers with a softmax objective. By learning representations end-to-end, x-vectors substantially outperformed i-vectors across multiple benchmarks.

Building on the TDNN backbone, Desplanques et al. proposed **ECAPA-TDNN** [4], which augments the x-vector paradigm with Squeeze-and-Excitation blocks for channel-wise attention, multi-scale feature aggregation, and attentive statistics pooling. Dawalatabad et al. [5] subsequently demonstrated that ECAPA-TDNN embeddings generalize well beyond verification to speaker diarization tasks. Training with the **ArcFace** angular-margin loss [6] further sharpens inter-speaker discrimination, yielding an Equal Error Rate (EER) of 0.87% on the **VoxCeleb1** test set [7]. Meanwhile, Jung et al. pursued a complementary direction with the **RawNet** family [8, 9], culminating in **RawNet3** [10], which operates directly on raw waveforms and eliminates the need for hand-designed spectral features altogether. These two architectures—ECAPA-TDNN and RawNet3—represent contrasting design philosophies (engineered features vs. learned front-ends) and form the basis of our comparative study.

2. Speaker Enrollment Strategies

Reliable speaker identification depends not only on the embedding model but also on the quality of the enrollment data used to construct each speaker's reference profile. Several lines of work have sought to improve enrollment.

Li et al. [11] proposed **enrollment speech augmentation**, applying noise and reverberation perturbations to enrollment utterances so that the resulting speaker template becomes more resilient to acoustic mismatch at test time. Mingote et al. [12] took a model-centric approach with **network-optimized enrollment models**: instead of averaging embeddings, they trained a per-speaker output head whose weights serve as the speaker template, eliminating the need for a separate scoring back-end. Le and Ngo [13] introduced a **data-centric enrollment selection** algorithm that iteratively retains only those utterances whose pairwise cosine similarities exceed an adaptive threshold, filtering out noisy or atypical recordings while preserving representative ones. Despite their effectiveness, all of these methods operate on the acoustic and environmental dimension of enrollment quality, largely ignoring the linguistic content of the enrolled speech.

3. Phonetic Information in Speaker Modeling

A parallel body of research has established that phonetic content plays a significant role in how well speaker characteristics are captured. Liu et al. [14, 15] conducted early investigations showing that **phoneme-aware speaker embeddings**—obtained by conditioning the network on phonetic labels—consistently improve verification accuracy over content-agnostic baselines. Wang et al. [16] corroborated this finding for text-independent scenarios, demonstrating that even implicit phonetic structure benefits embedding quality. Subsequent work has refined these ideas with more sophisticated architectures: Liu et al. [17] introduced a **phonetic attention mask** that guides the network to weight phonetically informative frames more heavily, while Hong et al. [18] proposed **phonetic decomposition and reorganization** modules that disentangle speaker and phonetic factors within the embedding space. Most recently, Song et al. [19] extended these techniques to the **multilingual** setting, showing that cross-lingual phonetic features transfer effectively for speaker verification across languages. Complementing these neural approaches, Dellwo et al. [20] provided acoustic-phonetic evidence that **prosodic and rhythmic variability** across a wider phonetic range better captures individual speaker traits.

In the enrollment domain, the **TIMIT** corpus [21]—a phonetically balanced speech database originally designed for ASR research—illustrates the long-recognized principle that controlled phonetic coverage leads to more informative speech samples. Yet, to the best of our knowledge, no prior work has explicitly leveraged phonetically rich, guided utterances during the enrollment phase for speaker identification. Le and Ngo [22] recently addressed this gap by proposing a **phoneme-based enrollment optimization** strategy that selects enrollment sentences to maximize phoneme coverage, thereby producing higher-quality speaker profiles. Their results demonstrate that bridging linguistic diversity with enrollment design leads to more robust and consistent recognition, and their approach provides a key foundation for the enrollment methodology adopted in this project.

III Datasets

While VoxCeleb [7] and VoxCeleb2 [23] remain the de facto benchmarks for speaker recognition, they are unsuitable for our benchmark for three key reasons. First, both candidate models were already pretrained on VoxCeleb; evaluating on the same corpus would confound downstream validation with pre-training exposure and fail to assess target-domain adaptation. Second, VoxCeleb's English-centric broadcast speech cannot evaluate robustness to *dialectal variation* across Vietnamese regions. Third, it lacks the typological breadth needed to test multilingual generalization. We therefore adopt two complementary datasets: the **ViMD** corpus [24] for fine-grained Vietnamese dialect coverage, and the **TidyVoice** corpus [25] for large-scale multilingual evaluation.

1. ViMD

The Vietnamese Multi-Dialect dataset (ViMD) [24], introduced at EMNLP 2024, contains approximately 102.56 hours of speech across roughly 19,000 utterances spanning all 63 provinces of Vietnam. Recordings are annotated with speaker identity codes (e.g., `spk_{province}_{id}`), gender labels, and province-level dialect tags. ViMD is valuable for our study because speakers from different provinces exhibit systematic phonetic and prosodic differences that a naïve model might conflate with speaker identity, requiring models to disentangle vocal characteristics from dialectal variation.

Table 1 summarizes the key statistics of the ViMD dataset as used in our experiments.

Property	Value
Total duration (hours)	≈102.56
Total utterances (processed)	18,947 (Train: 15,023, Val: 1,898, Test: 2,026)
Total speakers (processed)	12,953 (Train: 10,291, Val: 1,318, Test: 1,344)
Provinces / dialects	63 provinces
Sample format	16 kHz mono (resampled from 44.1/48 kHz)
Speaker leakage	0 (pruned overlapping singleton speakers)

Table 1: Summary statistics of the ViMD dataset splits.

2. TidyVoice

The TidyVoice dataset [25] is a multilingual corpus curated from Mozilla Common Voice with a rigorous speaker-identity cleaning pipeline that resolves the label noise inherent in raw Common Voice client IDs. It comprises two partitions:

- **Tidy-M** (monolingual): over 212,000 speakers across 81 languages.

- **Tidy-X** (cross-lingual): approximately 4,500 multilingual speakers across 40 languages, designed for cross-lingual speaker verification.

TidyVoice also underpins the **TidyVoice 2026 Challenge** at Interspeech 2026. It complements ViMD by testing whether speaker embeddings generalize across entirely different languages and recording environments—critical for globally deployed virtual assistants.

Table 2 presents the key statistics of the TidyVoice partition used in our experiments.

Property	**Value**
Source corpus	Mozilla Common Voice (Tidy-X partition)
Total utterances	321,711 (Train: 262,268, Val: 29,720, Test: 29,723)
Total speakers	4,474 (Train: 3,666, Val: 404, Test: 404)
Speaker overlap	0 (disjoint speaker sets across splits)
Sample format	16 kHz mono PCM
Split balancing	Language distribution & utterance count

Table 2: Summary statistics of the TidyVoice benchmark partition.

IV Model

This section describes the two speaker embedding architectures evaluated in our study. Both produce fixed-dimensional embeddings from variable-length audio but differ in their front-end design: ECAPA-TDNN operates on 80-bin Mel filterbank features, while RawNet3 learns directly from raw waveforms.

1. ECAPA-TDNN

ECAPA-TDNN, proposed by Desplanques et al. [4], enhances the x-vector TDNN architecture [2] with three innovations: SE-Res2Blocks, multi-layer feature aggregation, and channel-dependent attentive statistics pooling.

1.1 SE-Res2Block

The core building block (Figure 1) is a bottleneck residual unit. Two pointwise Conv1D layers sandwich a **Res2 dilated convolution**—a module from Res2Net that splits channels into $s = 8$ groups and processes them hierarchically, enabling multi-scale temporal modeling within a single layer. A **Squeeze-and-Excitation (SE)** module then recalibrates channels by computing global temporal statistics and learning per-channel scaling weights, effectively injecting utterance-level context into frame-level features. A skip connection completes the block.

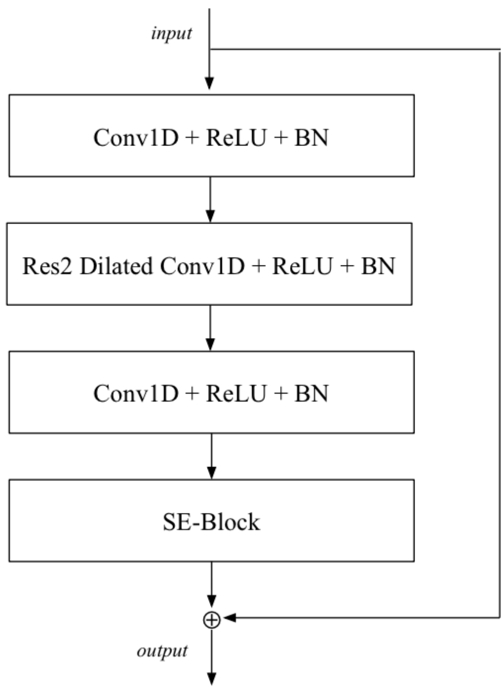

Figure 1: SE-Res2Block of ECAPA-TDNN [4].

1.2 Full Architecture

As shown in Figure 2, 80-dimensional Mel filterbank features pass through an initial Conv1D ($k = 5$), followed by three SE-Res2Blocks with increasing dilation ($d = 2, 3, 4$). The residual connection for each block is defined as the sum of *all* preceding block outputs, not just the immediate input.

The outputs of all three blocks are concatenated (**multi-layer feature aggregation**) and fused by a Conv1D ($k = 1$) into $1536 \times T$ features. A **channel-dependent attentive statistics pooling** layer computes per-channel attention weights—augmented with a global context vector (utterance-level mean and standard deviation)—to produce weighted means and standard deviations (3072×1). A final FC layer yields a 192-dim embedding, trained with AAM-Softmax [6].

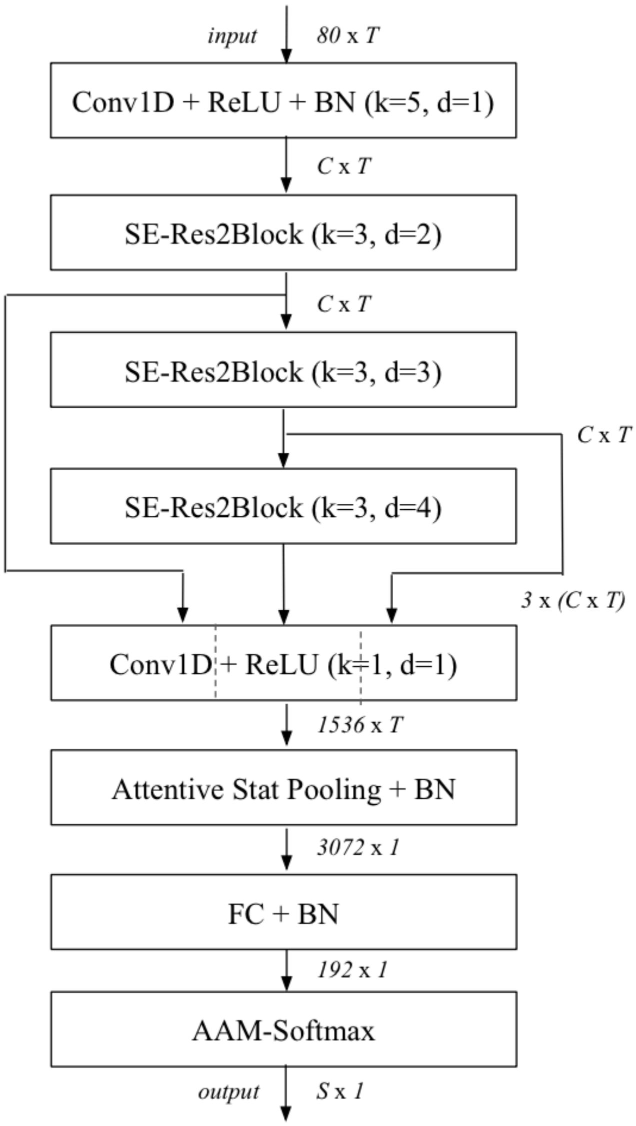

Figure 2: Full ECAPA-TDNN architecture [4].

2. RawNet3

RawNet3, proposed by Jung et al. [10], is the third generation of the RawNet family [8, 9]. It adopts ECAPA-TDNN's macro-topology but replaces the handcrafted front-end with a learnable one and adapts the building blocks for raw waveform input.

2.1 AFMS-Res2MP-Block

The backbone block (Figure 3) mirrors the SE-Res2Block with two modifications. First, **max pooling** is applied after the residual addition to reduce the much longer raw waveform sequences (pool sizes $p = 5$ and $p = 3$ in the first two blocks, while the third block applies no max pooling which is equivalent to $p = 1$). Second, **Adaptive Feature Map Scaling (AFMS)**—inherited from RawNet2—replaces the SE module, applying a learned affine channel-wise transformation based on global statistics.

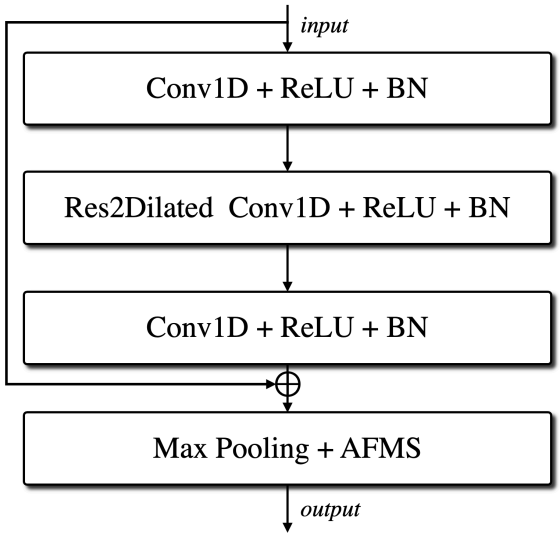

Figure 3: AFMS-Res2MP-block of RawNet3 [10].

2.2 Full Architecture

As shown in Figure 4, raw waveforms ($1 \times T$) first pass through pre-emphasis, instance normalization, and a **parameterized analytic filterbank**—a learnable Conv1D ($k = 251$, stride S) that replaces fixed acoustic-feature extraction, followed by log compression and mean normalization.

Three AFMS-Res2MP-blocks with parameters $(k, d, p) = (3, 2, 5),\ (3, 3, 3),\ (3, 4, 1)$, where $p = 1$ denotes no effective temporal downsampling in the third block. Following ECAPA-TDNN, all block outputs are concatenated and fused by a Conv1D. **Channel and context statistics pooling** (replacing RawNet2's GRU) collapses the temporal axis into 3072×1. A FC layer produces a 256-dim embedding, trained with AAM-Softmax.

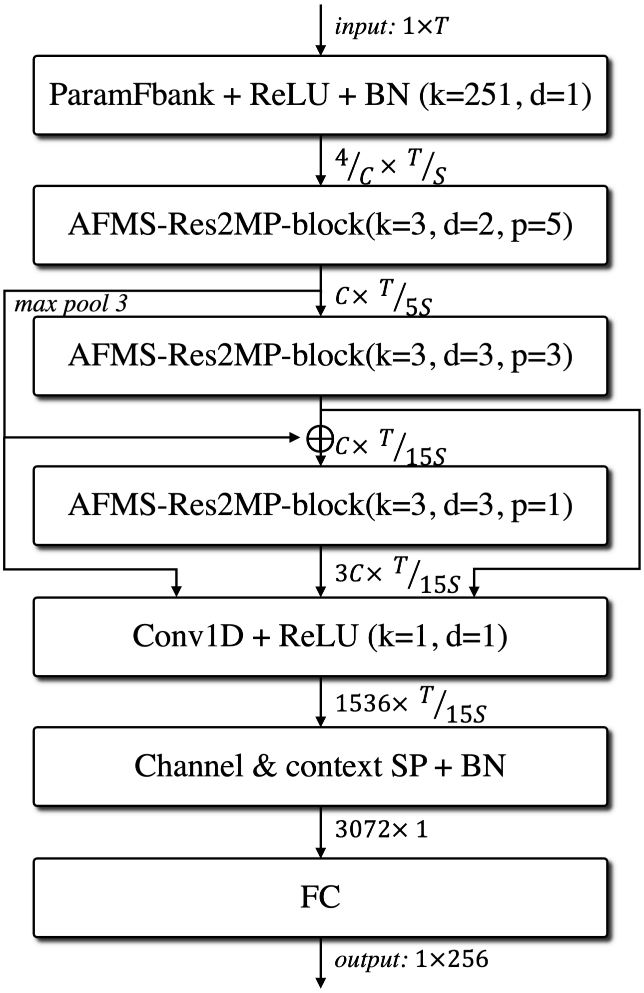

Figure 4: Full RawNet3 architecture [10].

3. Architectural Comparison

Property	ECAPA-TDNN	RawNet3
Input	80-dim Mel Featurebank feature	Raw waveform
Front-end	Fixed (handcrafted)	Learned (analytic fbank)
Core block	SE-Res2Block	AFMS-Res2MP-block
Channel attention	Squeeze-Excitation	AFMS
Sequence reduction	—	Max pooling
Pooling	Channel-dep. ASP	Channel & context SP
Embedding dim	192	256

Table 3: Comparison of ECAPA-TDNN and RawNet3.

V Experimental Methods

This section details our experimental methodology, including data preparation, fine-tuning protocols, and the evaluation framework used to benchmark ECAPA-TDNN and RawNet3 under resource-constrained conditions.

1. Data Partitioning and Preprocessing

To ensure strict zero-speaker leakage across splits, both datasets are partitioned into disjoint train, validation, and test subsets:

- **TidyVoice:** The official training split is preserved for training (262,268 utterances, 3,666 speakers). The source development set (808 speakers) is partitioned 50/50 into Validation (29,720 utterances, 404 speakers) and Test (29,723 utterances, 404 speakers) using a deterministic seed (seed 42), optimizing for balanced utterance counts and language distribution with zero speaker overlap.
- **ViMD:** The training split contains 15,023 utterances from 10,291 speakers. The original test split (2,026 utterances, 1,344 speakers) is kept intact. A data audit identified two overlapping singleton speakers between validation and test (`spk_73_0186` and `spk_76_0219`); these singleton rows were pruned from the validation set to eliminate speaker leakage, yielding a clean validation split of 1,898 utterances across 1,318 speakers without reducing the available genuine trial capacity.

Table 4 summarizes the standardized data splits across both corpora.

Dataset	Split	Utterances	Speakers	Genuine Pairs
TidyVoice	Train	262,268	3,666	—
	Validation	29,720	404	7,954
	Test	29,723	404	7,898
ViMD	Train	15,023	10,291	7,044
	Validation	1,898	1,318	863
	Test	2,026	1,344	1,042

Table 4: Summary of standardized dataset splits and trial capacities.

Audio Preprocessing Pipeline. All input audio files are standardized through a unified preprocessing pipeline:

1. Decode raw audio to 32-bit floating-point arrays.
2. Downmix multi-channel audio to mono via arithmetic averaging across channels.

3. Resample audio to 16 kHz using polyphase filtering.
4. Retain native acoustic dynamic range without amplitude normalization.
5. Deterministically crop utterances: for training, audio is cropped to 48,000 samples (3.00 s) for ECAPA-TDNN and 48,240 samples (3.015 s) for RawNet3; for evaluation, segments are cropped to 48,000 samples for ECAPA-TDNN and 64,240 samples (4.015 s) for RawNet3.
6. Utterances shorter than the target segment duration are extended via sample repetition rather than zero-padding to prevent introducing synthetic silence artifacts.

2. Fine-Tuning Configuration

Both models are initialized from publicly released, state-of-the-art checkpoints pretrained on VoxCeleb:

- **ECAPA-TDNN:** Loaded from `speechbrain/spkrec-ecapa-voxceleb` (revision `0f99f2d`), comprising 20.77M encoder parameters and outputting 192-dimensional embeddings.
- **RawNet3:** Loaded from `jungjee/RawNet3` (revision `c89102e`), comprising 16.28M encoder parameters and outputting 256-dimensional embeddings.

For target-domain adaptation, the original pretrained classification heads are discarded and replaced with freshly initialized Additive Angular Margin Softmax (AAM-Softmax) [6] heads, with output dimensions matching the number of training speakers in each respective dataset (3,666 for TidyVoice, 10,291 for ViMD). During downstream verification and identification inference, the classification heads are removed and only the L2-normalized encoder embeddings are utilized.

Configuration	ECAPA-TDNN	RawNet3
Random seed	42	42
Objective function	AAM-Softmax	AAM-Softmax
AAM margin (m) / scale (s)	0.2 / 30	0.2 / 30
Precision	FP16 (scale: 1024)	FP16 (scale: 1024)
Batch size	24	24
Optimizer	Adam	Adam
Encoder learning rate	1×10^{-4}	1×10^{-4}
Head learning rate	1×10^{-4}	1×10^{-4}
Weight decay	2×10^{-6}	5×10^{-5}
Gradient clipping	Global norm 5.0	Global norm 5.0
Max training epochs	3	3
Early stopping metric	Validation EER	Validation EER
Early stopping patience	1 epoch (tie: minDCF)	1 epoch (tie: minDCF)

Table 5: Hyperparameter settings and training configurations.

Resource-Constrained Protocol. To operate within the remaining project-time budget, training uses one deterministic utterance per speaker per epoch (3,666 examples/epoch for TidyVoice and 10,291 examples/epoch for ViMD). Memory calibration with the 10,291-class ViMD head tested batch sizes 4, 8, 16, 24, and 32; batch size 32 was the largest tested passing size, not a measured VRAM limit. The predefined rule selected a batch size no greater than 80% of that value, resulting in batch size 24, which was then confirmed by a three-batch real-audio optimization gate. Each architecture's two dataset runs were executed concurrently in a Kaggle T4$\times$2 session, with each Dataset assigned to it's single GPU.

3. Evaluation Protocol and Metrics

Verification performance is evaluated on frozen trial lists identical across all models to ensure an unbiased comparison:

- **Trial Generation:** To prevent speakers with large utterance counts from dominating the evaluation, genuine pairs are capped at a maximum of 20 pairs per speaker. Each evaluation split is paired with 100,000 randomly sampled impostor trials, providing statistical resolution down to a False Acceptance Rate (FAR) of 0.01%.
- **Scoring:** The trial score between an enrollment embedding e_1 and a test embedding e_2 is computed as the cosine similarity:

$$S(\boldsymbol{e}_1, \boldsymbol{e}_2) = \frac{\boldsymbol{e}_1 \cdot \boldsymbol{e}_2}{\|\boldsymbol{e}_1\|_2 \, \|\boldsymbol{e}_2\|_2} \tag{1}$$

- **Primary Metrics:** We report the Equal Error Rate (**EER** in %), where False Acceptance Rate equals False Rejection Rate (FAR $=$ FRR), and the minimum Detection Cost Function (**minDCF**) parameterized with $P_{\text{target}} = 0.01$ and $C_{\text{fa}} = C_{\text{miss}} = 1.0$:

$$C_{\text{Det}} = C_{\text{miss}} \cdot P_{\text{target}} \cdot \text{FRR} + C_{\text{fa}} \cdot (1 - P_{\text{target}}) \cdot \text{FAR} \tag{2}$$

- **Security Operating Point:** A security-critical threshold is calibrated on the Validation set at FAR $= 0.1\%$ and frozen. This threshold is then evaluated exactly once on the Test set to report operating accuracy, FAR, FRR, and True Acceptance Rate (TAR $= 1 -$ FRR). Additionally, we compute TAR@FAR across $\{5\%, 1\%, 0.1\%, 0.01\%\}$.

VI Results and Analysis

This section presents the empirical results of benchmarking ECAPA-TDNN and RawNet3 across TidyVoice and ViMD, detailing training dynamics, verification accuracy, and computational efficiency.

1. Training Dynamics and Convergence

Table 6 records the training progression across the four model–dataset pairs. Each run operated under an early stopping patience of 1 epoch on validation EER (with minDCF as tie-breaker).

Model / Dataset	Epoch	Train Loss	Val EER (%)	Val minDCF	Outcome
ECAPA-TDNN / TidyVoice	0	16.4067	**3.943**	**0.3685**	**Best Checkpoint**
	1	15.7529	4.075	0.4068	Early Stopped
ECAPA-TDNN / ViMD	0	17.0806	**3.936**	**0.4503**	**Best Checkpoint**
	1	15.3398	6.634	0.7661	Early Stopped
RawNet3 / TidyVoice	0	15.7133	**5.860**	**0.5251**	**Best Checkpoint**
	1	14.7825	9.693	0.7767	Early Stopped
RawNet3 / ViMD	0	15.9177	29.664	0.9164	Initialized
	1	14.8658	7.532	0.6805	Improved
	2	13.3438	**2.692**	**0.2446**	**Best Checkpoint**

Table 6: Training progression and checkpoint selection across four experimental runs.

A notable observation is that decreasing training loss did not consistently translate into lower validation error. For ECAPA-TDNN on both datasets and RawNet3 on TidyVoice, the models achieved their best validation metrics at Epoch 0; subsequent epochs caused validation degradation, triggering early stopping. In contrast, RawNet3 on ViMD exhibited continuous and substantial convergence, reducing validation EER from 29.66% at Epoch 0 to 2.69% at Epoch 2.

2. Speaker Verification Benchmark

Table 7 reports the primary evaluation metrics on the unseen Test splits using the best checkpoints selected on Validation.

Model	Dataset	EER (%) ↓	minDCF ↓	Accuracy (%)	FAR (%)	FRR (%)	TAR (%) ↑
ECAPA-TDNN	TidyVoice	**3.912**	**0.3586**	97.865	0.080	28.159	**71.841**
RawNet3	TidyVoice	5.875	0.5624	96.509	0.124	46.126	53.874
ECAPA-TDNN	ViMD	3.804	0.4516	99.527	0.116	34.741	65.259
RawNet3	ViMD	**3.071**	**0.2262**	**99.765**	**0.075**	**15.547**	**84.453**

Table 7: Final test performance on TidyVoice and ViMD. Thresholds are calibrated at FAR $= 0.1\%$ on the Validation split and evaluated once on the Test split.

Analysis of Operating Performance. High overall accuracy values ($> 96\%$) reflect the protocol's 100,000 impostor trials relative to genuine trials. Therefore, EER, minDCF, and True Acceptance Rate (TAR) at low False Acceptance Rates serve as the discriminative metrics:

- **On TidyVoice (Multilingual):** ECAPA-TDNN achieves superior performance across all metrics, reducing EER by 33.4% relative (3.912% vs. 5.875%) and minDCF by 36.2% relative (0.3586 vs. 0.5624), while yielding a +17.97 percentage points higher TAR at the frozen security threshold.
- **On ViMD (Vietnamese Multi-Dialect):** RawNet3 outperforms ECAPA-TDNN, achieving a 19.3% relative reduction in EER (3.071% vs. 3.804%) and a 49.9% relative reduction in minDCF (0.2262 vs. 0.4516). At the operational threshold, RawNet3 boosts TAR to 84.453% (compared to 65.259% for ECAPA-TDNN) while maintaining a strict 0.075% FAR.

Table 8 provides a fine-grained evaluation of authentication security, detailing the True Acceptance Rate at varying target False Acceptance Rates (TAR@FAR).

Model	Dataset	TAR@FAR 5%	TAR@FAR 1%	TAR@FAR 0.1%	TAR@FAR 0.01%
ECAPA-TDNN	TidyVoice	**96.771**	**90.377**	**73.563**	**48.974**
RawNet3	TidyVoice	93.391	82.907	48.088	0.886
ECAPA-TDNN	ViMD	**97.505**	88.484	63.820	37.812
RawNet3	ViMD	97.217	**95.106**	**86.756**	**54.798**

Table 8: True Acceptance Rate (%) at specific False Acceptance Rate thresholds (TAR@FAR).

3. Computational Efficiency and Model Selection

To evaluate suitability for real-time deployment in our virtual assistant pipeline, standalone model inference latency was benchmarked on an NVIDIA Tesla T4 GPU using CUDA events across 50 iterations with batch size 1 after 10 warm-up runs (Table 9).

Model	Parameters	Median Latency (ms)	p95 Latency (ms)
ECAPA-TDNN	20.77 M	12.023	12.912
RawNet3	**16.28 M**	**8.840**	**12.884**

Table 9: Model complexity and isolated GPU inference latency (batch size 1 on NVIDIA Tesla T4).

Deployment Selection Rationale. Model selection is based on Validation evidence and deployment constraints; Test results are used only for final confirmation. **RawNet3 fine-tuned on ViMD (Epoch 2)** is selected as the verification engine for the Who-Speak-AI virtual assistant:

1. **Validation Evidence and Target-Domain Alignment:** The intended application targets Vietnamese speech, making ViMD the priority domain. On ViMD Validation, RawNet3 achieved 2.692% EER and 0.2446 minDCF, outperforming ECAPA-TDNN at 3.936% EER and 0.4503 minDCF.
2. **Final Test Confirmation:** With the Validation-selected checkpoint and frozen Validation threshold, ViMD Test confirms the same ranking: RawNet3 achieved 3.071% EER, 0.2262 minDCF, 0.075% FAR, and 84.453% TAR.
3. **Efficiency Profile:** RawNet3 contains 21.6% fewer parameters (16.28M vs. 20.77M) and exhibits 26.5% lower median GPU latency (8.84 ms vs. 12.02 ms), preserving critical compute headroom for concurrent ASR, LLM reasoning, and TTS synthesis.

4. Limitations

The experiment face multiple limitations: single-seed training (42), three epochs with early stopping, deterministic utterance per speaker per epoch; No hyperparameter grid search, margin ablation, scheduler ablation, augmentation study, multi-seed variance estimate, or CPU benchmark was performed.

The reported latency is model-only latency on a Tesla T4 and excludes audio decoding, resampling, application logic, networking, ASR, and TTS. The security threshold was calibrated on ViMD rather than live application microphones and therefore serves only as an evidence-based initial value.

WavLM+MHFA was implemented and attempted but excluded from the final comparison because no complete accepted run passed the evidence validator: one attempt produced non-finite FP16 training embeddings, and a later attempt produced non-finite evidence during ViMD Final Test. No WavLM checkpoint or metric is reported as an accepted result.

VII Application: Who-Speak-AI

This section presents *Who-Speak-AI*, an end-to-end conversational virtual assistant that integrates our fine-tuned RawNet3 speaker verification model into a modular, privacy-preserving voice pipeline.

1. Architectural Design

Who-Speak-AI adopts a decoupled multi-service architecture (Figure 5):

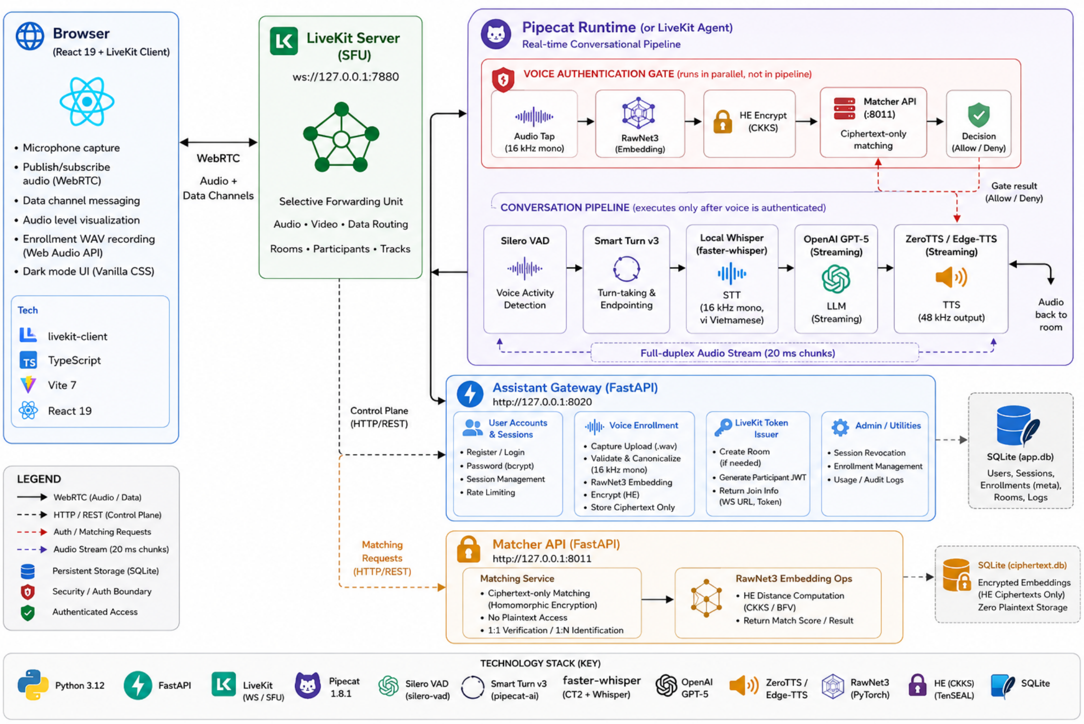

Figure 5: System architecture of Who-Speak-AI

1.1 User Interface

The presentation layer comprises a production web client and an offline diagnostic dashboard: **Production Web Application (app/web):** Built with React 19, TypeScript 5.7, Vite 7, and Vanilla CSS (dark-mode, glassmorphism). The UI coordinates four core functions: (1) *Coordinator* for registration, login (`scrypt`, `HttpOnly` cookies), and 3-sample voice enrollment; (2) *Voice Stage* rendering an animated orb tracking RMS audio energy across states (*Idle*, *Listening*, *Thinking*, *Speaking*, *Challenging*); (3) *Conversation Feed* displaying streaming transcripts, authorization badges, tool receipts, and biometric challenge modals; and (4) *Hardware Telemetry* managing I/O devices and WebRTC status. **Client-Side Audio Engineering:** Audio is captured via Web Audio API, downmixing to mono 16-bit linear PCM at 16 kHz. Full-duplex WebRTC streaming is handled by `livekit-client` 2.22.0 across topics `voice-auth`, `voice-auth-status`, `voice-agent-event`, and `voice-agent-command`. **Diagnostic Dashboard:** A Streamlit app for offline 1:1/1:N verification, waveform/spectrogram inspection, and ROC/DET calibration.

1.2 Runtimes

System & Media Transport: Python 3.12 backend and Node.js frontend. Media routing is handled by **LiveKit Server** (SFU) with 16 kHz mono PCM ingress and 48 kHz mono PCM egress in the Pipecat transport layer. **Conversational Pipeline (Pipecat):** Orchestrated by **Pipecat** in a framed asynchronous flow:

$$\text{LiveKit In} \longrightarrow \text{AuthRouter} \longrightarrow \text{VAD} \longrightarrow \text{Smart Turn} \longrightarrow \text{STT} \longrightarrow \text{LLM} \longrightarrow \text{TTS} \longrightarrow \text{LiveKit Out} \tag{3}$$

A `UserBotLatencyObserver` continuously profiles user-to-bot conversational latency across stages. **Session Supervisor:** A FastAPI daemon (port 8021) spawns per-room workers via HMAC-SHA256 signed session descriptors from the Gateway, enforcing a 45 s join timeout and managing room lifecycle. **Alternate Worker Runtime:** Legacy standalone **LiveKit Agents** (≥ 1.6) is retained as an MCP-compatible fallback, mutually exclusive with Pipecat per room. **Microservices Architecture:** Two FastAPI microservices divide application logic: (1) *Assistant Gateway* (port 8020) manages accounts, sessions, enrollment, and signed LiveKit tokens; (2) *Matcher API* (port 8011) stores public CKKS contexts and encrypted voiceprints in SQLite, evaluating ciphertext-only squared Euclidean distances without accessing raw audio, plaintext embeddings, secret keys, or plaintext scores.

1.3 Model and Agent

The intelligence and security tier integrates neural acoustic modeling, endpointing, language reasoning, and homomorphic speaker verification (Table 10).

Subsystem	Model / Tech	Runtime	Operational Configuration
VAD	Silero VAD v5	Pipecat / PyTorch	$p \geq 0.7$, 384 ms onset, 650 ms silence, 20 s max utterance.
Turn-Taking	Smart Turn v3 (`onnx`)	ONNX Runtime ≥ 1.17	Semantic classifier; suppresses premature interruption (max +2 s).
Speech-to-Text	PhoWhisper-small-ct2	CTranslate2 int8 / CPU	16 kHz mono Vietnamese (`vi`); local in-memory decoding.
LLM Reasoning	OpenAI (`OPENAI_MODEL`)	AsyncOpenAI	Streaming dialogue agent (default `gpt-5`); isolated from biometrics.
Chunking	SentenceBuffer	Custom Regex	Punctuation boundary slicing (≥ 120 chars) for low TTFA.
Neural TTS	ZeroTTS (`maichi`)	ONNX Runtime 0.1.1	Local 48 kHz mono neural stream; fallback to Azure Edge-TTS.
Speaker Verif.	RawNet3 (ViMD fine-tuned)	PyTorch / Sinc-Conv	16.28M params; 256-D L_2-normalized raw waveform embedding.
Encryption	TenSEAL (CKKS scheme)	TenSEAL 0.3.16	$N = 8192, Q = [60, 40, 40, 60], \Delta = 2^{40}$; 128-bit security.
Key Protection	OS Hardware Keychain	`keyring` 25.x	Stores CKKS secret key sk; fails closed if unavailable.
Tool Policy	PolicyToolExecutor	Native Python	State-gated execution; `MockCalendar` or `GatewayCalendar` (MCP).

Table 10: Technical inventory of models, runtimes, and functional subsystems in Who-Speak-AI.

Voice Activity Detection (Silero VAD v5). The model computes speech probabilities per 20 ms frame from incoming 16 kHz 16-bit linear PCM audio frames:

- **Vocal Onset:** Speech is confirmed once frame probability satisfies $p \geq 0.7$ continuously for at least 384 ms (`VOICE_VAD_MIN_SPEECH_SECONDS=0.384`).

- **Vocal Offset:** Silence is recognized after 650 ms below threshold (`VOICE_VAD_SILENCE_SECONDS=` with a hard maximum utterance cap of 20 s (`VOICE_VAD_MAXIMUM_SECONDS=20`).

Turn-Taking and Endpointing (Smart Turn v3). Conventional fixed-silence timers prematurely interrupt speakers during natural cognitive pauses. Who-Speak-AI integrates Smart Turn v3 (`smart_turn_v3.onnx`) running on ONNX Runtime (≥ 1.17). When Silero signals a silence boundary, Smart Turn ingests the buffered audio and runs an end-of-turn classifier evaluating prosodic, acoustic, and linguistic completion cues. If the turn is predicted to be ongoing, cut-off is suppressed for up to an additional 2 s (`VOICE_SMART_TURN_MAX_WAIT_SECONDS=2`) An in-memory RMS turn buffer (`LocalSpeechTurnBuffer`, amplitude threshold 0.012) provides a deterministic fallback.

Speech-to-Text (PhoWhisper). User speech is transcribed locally using `faster-whisper` with the `vudang449/PhoWhisper-small-ct2` checkpoint—a Vietnamese-specialized CTranslate2 model running in `int8` quantization on CPU (`VOICE_WHISPER_MODEL`, `VOICE_WHISPER_COMPUT` Operating at 16 kHz mono with a forced language prior (`language="vi"`), the engine decodes audio directly in system memory without dispatching waveforms or transcripts to any external cloud service.

Conversational Reasoning (OpenAI LLM). Clean transcripts are dispatched to OpenAI via the streaming Responses API. The model is configured via `OPENAI_MODEL` (default `gpt-5`). The cognitive core is strictly decoupled from Voice Verification data:

- **Deterministic Capability Policy:** Dialogue policy is governed by `ConversationSupervisor` and `SupervisorPolicy`. The LLM is restricted to generating natural-language text. Tool execution orchestrated explicitly to verify user and inject only accessible capability signatures (`allowed_tools`).
- **Token Chunking:** To minimize Time-to-First-Audio (TTFA) latency, text is cleanly partitioned at the nearest preceding word boundary and sent to TTS, then LLM generation with audio synthesis.

Neural Speech Synthesis. Audio vocalization uses a dual-engine architecture:

- **Primary Engine - ZeroTTS:** A local neural TTS model running under ONNX Runtime with multi-threaded inference. Using preset Vietnamese weights (`maichi`), it synthesizes 48 kHz mono audio with a 150–250 ms startup buffer.
- **Fallback Engine - EdgeTTS:** If ZeroTTS encounters initialization or hardware failures, the pipeline dynamically fails over to Microsoft Edge-TTS, streaming Azure's `vi-VN-HoaiMyNeural` voice.

Biometric Feature Extraction (RawNet3). Biometric identity verification is powered by our fine-tuned RawNet3 model (16.28M parameters). RawNet3 processes raw 16 kHz mono PCM waveforms (≥ 3.0 s) directly in the time domain:

1. **Time-Domain Feature Extraction:** A parameterized Sinc-convolution filterbank replaces standard Short-Time Fourier Transform (STFT) or log-mel spectrogram computation, preserving phase information.

2. **Deep Representation:** Filterbank outputs pass through a modified Res2Net architecture enhanced with Alpha-Feature-Map Scaling (AFMS) and Squeeze-and-Excitation (SE) channel blocks.

3. **Embedding Normalization:** The model outputs a continuous 256-dimensional speaker embedding $\mathbf{v} \in \mathbb{R}^{256}$, normalized under the L_2-norm:

$$\mathbf{v} \leftarrow \frac{\mathbf{v}}{\|\mathbf{v}\|_2}, \qquad \|\mathbf{v}\|_2 = 1 \tag{4}$$

Encrypted Matching & Hardware Key Protection. The candidate embedding $\mathbf{v}$ is verified under TenSEAL's CKKS homomorphic encryption scheme ($N = 8192, Q = [60, 40, 40, 60], \Delta = 2^{40}$). The secret key sk is sealed inside the operating system keychain via Python `keyring` under service `who-speak.voice-he`. During verification, the Matcher API evaluates the homomorphic inner product $[\![s]\!] = [\![\mathbf{u}]\!] \odot \mathbf{v}$ over encrypted reference vector $[\![\mathbf{u}]\!]$. The client or agent decrypts $s = \mathsf{Dec}_{\mathsf{sk}}([\![s]\!])$. Verification succeeds if:

$$s \geq \tau \approx 0.657 \quad \text{(configurable via VOICE\_SV\_THRESHOLD)} \tag{5}$$

Audio Gate Isolation (`AuthRouterProcessor`). `AuthRouterProcessor` is the *first* processor in the Pipecat pipeline, positioned before Silero VAD and Whisper STT. In phases `CAPTURING`, `PROCESSING`, and `WAITING_FOR_RESUME`, incoming audio frames are consumed or discarded by the router and never forwarded downstream. Only when the phase is `IDLE` or `CONVERSATION_READY` (and a brief resume-guard window has elapsed) does the router forward frames to VAD and the conversation pipeline. This guarantees that the spoken authentication phrase is never processed by VAD, never transcribed by Whisper, and never ingested by the LLM.

Policy-Gated Tool Execution. System capabilities are divided into public utilities accessible in guest mode and protected personal tools (e.g., private calendar access). Guest users can freely engage in general-purpose conversation; privileged tool calls require AuthState = `AUTHENTICATED`. Unauthenticated invocations trigger a voice challenge. After the challenge result is available, the user must explicitly press *Resume* or *Continue as Guest*—this action opens the conversation ingress. Upon successful cryptographic verification ($s \geq \tau$), a session authorization TTL (default 5 minutes) is activated, permitting deterministic tool execution. The calendar backend is `MockCalendarProvider` when `MCP_PROVIDER=mock`, or `GatewayCalendarProvider` (proxying to a local or Google MCP server) when `MCP_PROVIDER=local` or `mcp`.

Audio Gate Isolation Boundary

A critical security vulnerability in voice-driven assistants is "passphrase leakage," where sensitive authentication speech is transcribed by ASR into dialogue history. Who-Speak-AI eliminates this via `AuthRouterProcessor`: challenge audio is intercepted at the WebRTC transport boundary before VAD or Whisper STT. Challenge frames are routed strictly to RawNet3 and CKKS matching, guaranteeing zero biometric or linguistic leakage into LLM contexts.

2. Privacy-Preserving Voice Authentication

2.1 CKKS Scheme Formulation

Who-Speak-AI implements the Cheon-Kim-Kim-Song (CKKS) cryptosystem [26] via the TenSEAL library [27]. CKKS supports homomorphic addition and multiplication on vectors of complex numbers, making it well-suited for fixed-dimensional continuous embedding spaces.

The cryptosystem is configured with polynomial modulus degree $N = 8192$, coefficient modulus bit sizes $Q = [60, 40, 40, 60]$, and scaling factor $\Delta = 2^{40}$. This parameter set guarantees 128-bit post-quantum and classical security while accommodating the multiplicative depth required for inner product evaluation.

2.2 Encrypted Cosine Similarity Evaluation

Let $\mathbf{u} \in \mathbb{R}^{256}$ denote the enrolled reference embedding and $\mathbf{v} \in \mathbb{R}^{256}$ denote the candidate verification embedding extracted by RawNet3. Both vectors are L_2-normalized prior to encryption:

$$\|\mathbf{u}\|_2 = 1, \quad \|\mathbf{v}\|_2 = 1 \implies \cos(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^{256} u_i v_i \tag{6}$$

During enrollment, the client generates a public context containing the public key pk, relinearization keys rlk, and Galois rotation keys gk, while retaining the secret key sk in the OS keychain. The reference vector $\mathbf{u}$ is encrypted into ciphertext $[\![\mathbf{u}]\!] \leftarrow \mathsf{Enc}_{\mathsf{pk}}(\mathbf{u})$ and transmitted to the Matcher service.

During verification, the Matcher API computes a homomorphic squared Euclidean distance between the stored enrolled ciphertext $[\![\mathbf{u}]\!]$ and the candidate query ciphertext $[\![\mathbf{v}]\!]$:

$$[\![d]\!] = ([\![\mathbf{v}]\!] - [\![\mathbf{u}]\!]) \cdot ([\![\mathbf{v}]\!] - [\![\mathbf{u}]\!]) \tag{7}$$

where subtraction and dot product are applied homomorphically in ciphertext space. The encrypted scalar $[\![d]\!]$ is returned to the trusted client or agent supervisor, which decrypts:

$$d = \mathsf{Dec}_{\mathsf{sk}}([\![d]\!]) \tag{8}$$

Verification succeeds if $d \leq \tau_d$, corresponding to threshold $\tau \approx 0.657$ on the underlying cosine similarity space (`SV_THRESHOLD     0.65653`; configurable via `VOICE_SV_THRESHOLD`). At no point does the Matcher observe plaintext embeddings, raw audio, sk, or the plaintext score d.

Homomorphic Zero-Trust Guarantee

Traditional systems must decrypt biometrics into server memory before computing distances, creating an exposure window. Under Who-Speak-AI's CKKS architecture, the Matcher API receives only ciphertexts and public context—it never receives raw audio, plaintext embeddings, the secret key sk, or the plaintext score s. Note that the trusted local voice-auth service does hold the waveform and embedding transiently during extraction and encryption; the zero-knowledge guarantee applies specifically to the untrusted Matcher microservice boundary.

3. Verifying of Enrollment Quality

While advanced enrollment quality verification—such as data-centric utterance filtering [13] and phoneme-balanced selection [22]—was planned to ensure robust reference templates, we candidly acknowledge that it has not been implemented due to project time constraints.

4. Latency

End-to-end latency (VAD-detected end of speech to first audible TTS output) was measured over $N = 10$ trials by stopwatch. Table 11 reports the aggregate statistics.

Metric	Value
Trials (N)	10
Mean ($\bar{t}$)	7.5 s
Std. dev. (σ)	2.1 s
Variance (σ^2)	4.4 s^2

Table 11: End-to-end latency summary (manual stopwatch, $N = 10$; raw trial data unavailable).

The elevated mean latency (7.5 s) and standard deviation ($\sigma = 2.1$ s) reflect the cumulative overhead of the local processing chain: Whisper `base` CPU inference, OpenAI GPT-5 network round-trip, ZeroTTS ONNX warmup and synthesis, and LiveKit media routing. The large spread is consistent with non-deterministic CPU scheduling and network jitter under a local development environment. A dedicated latency profiling run with automated timestamps (e.g., via the `UserBotLatencyObserver` already instrumented in the Pipecat runtime) is left as future work.

VIII Conclusion

This technical report benchmarked on two Speaker Embedding models: ECAPA-TDNN and RawNet3 under TidyVoice Vietnamese multi-dialectal (ViMD) corpora. On TidyVoice, ECAPA-TDNN demonstrated better cross-lingual generalization (3.912% EER, 0.3586 minDCF, and 71.84% TAR@0.1%). However, when adapted to regional Vietnamese speech on ViMD, RawNet3 proved more efficacy at disentangling dialect from identity (3.071% EER, 0.2262 minDCF, and 84.45% TAR), which also comes with the benefit of smaller parameter footprint. Thus, RawNet3 was selected as the biometric engine for our Vietnamese target application.

Translating these empirical findings into practice, we developed *Who-Speak-AI*, an end-to-end voice assistant integrating RawNet3 into a modular pipeline featuring Whisper ASR, LLM dialogue reasoning, and neural TTS. The system couples RawNet3 embeddings with the CKKS homomorphic encryption scheme, allowing server-side cosine matching to execute strictly over encrypted ciphertexts. In our experiments, this approach maintains high accuracy while protecting user privacy.

Nevertheless, several limitations remain: **(1)** only two models were benchmarked, as WavLM + MHFA failed during training and testing; **(2)** fine-tuning was constrained to a single seed (42) and 3 epochs without exhaustive tuning; **(3)** thresholds were calibrated on static splits rather than live streams; and **(4)** the system with homomorphic operations currently CPU-bound. Future work will explore anti-spoofing and hardware acceleration for encrypted matching.

References

[1] N. Dehak, P. J. Kenny, R. Dehak, P. Dumouchel, and P. Ouellet, "Front-end factor analysis for speaker verification," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 19, no. 4, pp. 788–798, 2010.

[2] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, "X-vectors: Robust DNN embeddings for speaker recognition," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 5329–5333.

[3] V. Peddinti, D. Povey, and S. Khudanpur, "A time delay neural network architecture for efficient modeling of long temporal contexts," in *Interspeech*, 2015, pp. 3214–3218.

[4] B. Desplanques, J. Thienpondt, and K. Demuynck, "ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification," *Proc. Interspeech*, pp. 3830–3834, 2020.

[5] N. Dawalatabad, M. Ravanelli, F. Grondin, J. Thienpondt, B. Desplanques, and H. Na, "ECAPA-TDNN embeddings for speaker diarization," in *Interspeech 2021*, 2021, pp. 3560–3564.

[6] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4690–4699.

[7] A. Nagrani, J. S. Chung, and A. Zisserman, "VoxCeleb: A large-scale speaker identification dataset," in *Interspeech 2017*, 2017, pp. 2616–2620.

[8] J.-w. Jung, H.-S. Heo, J.-h. Yang, H.-j. Shim, and H.-J. Chung, "RawNet: Advanced end-to-end deep neural network using raw waveforms for text-independent speaker verification," *Proc. Interspeech*, pp. 1268–1272, 2019.

[9] J.-w. Jung, H.-S. Heo, J.-h. Yang, H.-j. Shim, and H.-J. Chung, "RawNet2: Direct raw waveform inputs for speaker verification," *Proc. Interspeech*, pp. 1560–1564, 2020.

[10] J.-w. Jung, Y. J. Kim, H.-S. Heo, B.-J. Lee, Y. Kwon, and J. S. Chung, "Pushing the limits of raw waveform speaker recognition," *Proc. Interspeech*, pp. 2213–2217, 2022.

[11] J. Li, K. Zhang, S. Wang, H. Li, M.-W. Mak, and K. A. Lee, "On the effectiveness of enrollment speech augmentation for target speaker extraction," in *2024 IEEE Spoken Language Technology Workshop (SLT)*, 2024, pp. 325–332.

[12] V. Mingote, A. Miguel, A. Ortega Giménez, and E. Lleida, "Training speaker enrollment models by network optimization," in *INTERSPEECH*, 2020, pp. 3810–3814.

[13] L.-Q. Le and M.-N. Ngo, "Data-centric optimization of enrollment selection in speaker identification," in *Proceedings of the 14th International Conference on Pattern Recognition Applications and Methods – ICPRAM*. SciTePress, 2025, pp. 344–351.

[14] Y. Liu, L. He, J. Liu, and M. T. Johnson, "Speaker embedding extraction with phonetic information," in *Interspeech 2018*, 2018, pp. 2247–2251.

[15] Y. Liu, L. He, J. Liu, and M. T. Johnson, "Introducing phonetic information to speaker embedding for speaker verification," *EURASIP Journal on Audio, Speech, and Music Processing*, 2019.

[16] S. Wang, J. Rohdin, L. Burget, O. Plchot, Y. Qian, K. Yu, and J. Cernocký, "On the usage of phonetic information for text-independent speaker embedding extraction," in *Interspeech*, 2019, pp. 1148–1152.

[17] T. Liu, R. K. Das, K. A. Lee, and H. Li, "Neural acoustic-phonetic approach for speaker verification with phonetic attention mask," *IEEE Signal Processing Letters*, vol. 29, pp. 782–786, 2022.

[18] Q.-B. Hong, C.-H. Wu, and H.-M. Wang, "Decomposition and reorganization of phonetic information for speaker embedding learning," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1745–1757, 2023.

[19] Z. Song, L. He, P. Wang, Y. Hu, and H. Huang, "Introducing multilingual phonetic information to speaker embedding for speaker verification," in *ICASSP 2024 – 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 10 091–10 095.

[20] V. Dellwo, A. Leemann, and M.-J. Kolly, "Rhythmic variability between speakers: Articulatory, prosodic, and linguistic factors," *The Journal of the Acoustical Society of America*, vol. 137, p. 1513, 2015.

[21] J. Garofolo, L. Lamel, W. Fisher, J. Fiscus, and D. Pallett, "DARPA TIMIT acoustic-phonetic continuous speech corpus CD-ROM. NIST speech disc 1-1.1," *NASA STI/Recon Technical Report N*, vol. 93, p. 27403, 1993.

[22] L.-Q. Le and M.-N. Ngo, "Phoneme-based optimization of enrollment selection for speaker identification," in *PRICAI 2025: Trends in Artificial Intelligence*. Springer Nature Singapore, 2026, pp. 523–536.

[23] J. S. Chung, A. Nagrani, and A. Zisserman, "VoxCeleb2: Deep speaker recognition," in *Interspeech 2018*, 2018, pp. 1086–1090.

[24] N. V. Dinh, T. C. Dang, L. T. Nguyen, and K. V. Nguyen, "Multi-dialect Vietnamese: Task, dataset, baseline models and challenges," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 7679–7693.

[25] A. H. Farhadipour *et al.*, "TidyVoice: A curated multilingual dataset for speaker verification derived from Common Voice," *arXiv preprint arXiv:2601.16358*, 2026.

[26] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer International Publishing, 2017, pp. 409–437.

[27] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal, "TenSEAL: A library for encrypted tensor operations using homomorphic encryption," *arXiv.org*, 2021.

[28] E. Marchi, V. Ganapathi-Subramanian, S. Kenealy, J. Pinto, and I. Bulyko, "Generalised discriminative feature representations for speaker recognition in Siri," *Proc. IEEE Spoken Language Technology Workshop (SLT)*, pp. 837–844, 2018.

[29] S. Garimella, C.-C. Weng, and N. Strom, "Robust speaker identification for personalized speech recognition," *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 7544–7548, 2020.